

基于树结构与深度强化学习的在线三维装箱算法

张长勇*, 张宇浩, 李铮
(中国民航大学, 天津 300300)

摘要: **目的** 针对现有在线三维装箱算法在应对动态、复杂及大规模装箱场景时普遍存在的计算效率低、学习空间易爆炸等问题, 提出一种融合树结构与深度强化学习 (DRL) 的在线装载方法。 **方法** 构建了分层耦合优化框架: 在外层, 引入缓冲区与临时备份空间, 利用树结构规划器生成包含货物选取、装载及“移除重排”的动作序列, 突破传统方法无法修正历史决策的局限; 在内层, 设计拓扑-空间融合感知网络, 结合卷积神经网络 (CNN) 与图卷积网络 (GCN) 提取容器几何剩余空间与货物支撑结构特征, 利用 DRL 智能体输出最优落位并评估状态价值, 引导树搜索进行高效剪枝。 **结果** 实验显示, 在考虑物理支撑与稳定性等复杂现实约束下, 该方法较现有主流 DRL 算法在容器的空间利用率上提升约 15%, 并在 50~200 件大规模货物序列下保持了稳定的实时响应能力。 **结论** 所提算法有效实现了在线装箱中长序列规划与局部空间优化的平衡, 具有显著的工程应用价值。

关键词: 在线三维装箱; 树结构规划; 深度强化学习; 拓扑感知; 神经网络

中图分类号: V353; TB181

文献标志码: A

文章编号: 1001-3563(2026)05-0130-14

DOI: 10.19554/j.cnki.1001-3563.2026.05.015

An Online 3D Packing Algorithm Based on Tree Structures and Deep Reinforcement Learning

ZHANG Changyong*, ZHANG Yuhao, LI Zheng

(Civil Aviation University of China, Tianjin 300300, China)

ABSTRACT: The work aims to propose an online loading method integrating tree structures with deep reinforcement learning (DRL) so as to address the prevalent issues of low computational efficiency and prone-to-exploding learning space in existing online 3D packing algorithms when handling dynamic, complex, and large-scale packing scenarios. A hierarchical coupled optimisation framework was constructed. At the outer layer, a buffer zone and a temporary backup space were introduced. A tree-structured planner was applied to generate action sequences encompassing cargo selection, loading, and "remove-and-rearrange" operations, overcoming the limitation of traditional methods that could not correct historical decisions. At the inner layer, a topology-space fusion perception network was designed. Combining convolutional neural networks (CNN) and graph convolutional networks (GCN), the container geometric residual space and cargo support structure features were extracted. The DRL agent was applied to output optimal placement positions and evaluate state values, guiding tree search for efficient pruning. Experiments demonstrated that under complex real-world constraints such as physical support and stability, this method achieved approximately 15% higher container space utilisation than existing mainstream DRL algorithms, while maintaining stable real-time response capabilities across

收稿日期: 2025-10-23

基金项目: 中央高校基本科研业务费-高水平成果培育项目 (3122025TD08); 中国交通教育研究会教育科学研究重点课题 (JT2024ZD066)

*通信作者

large-scale cargo sequences ranging from 50 to 200 items. The proposed algorithm effectively balances long-sequence planning with local space optimisation in online container loading, demonstrating significant engineering application value.

KEY WORDS: online 3D packing; tree structure planning; deep reinforcement learning; topology awareness; neural networks

“智慧物流”正在推动物流领域中仓储、运输、分拨、配送等新一代环节的变革,同时也对这些环节的成本降低与效率提升提出了更为严苛的要求。其中,运输装载环节是实现成本降低的核心所在,如何高效、合理地利用容器内部有限的空间,以最大化货物装载量,减少运输成本,已经成为物流企业亟待解决的关键问题。

货物装载问题归属于三维装箱问题(3D Bin Packing Problem, 3D-BPP),属于 NP-Hard 问题^[1]。根据货物序列是否已知可以将三维装箱问题分为离线三维装箱和在线三维装箱两部分。传统的三维装箱问题通常采用精确算法^[2-3]、启发式算法^[4-5]、元启发式算法^[6-7]和树搜索算法^[8-9]等方法进行求解,针对离线三维装箱问题的研究有一定成果。然而,这些方法往往存在计算效率低、求解结果不稳定等问题,尤其在面对动态、复杂和大规模的在线装箱问题时,难以得到令人满意的解决方案。

近年来,随着深度学习和强化学习的快速发展,基于深度强化学习(Deep Reinforcement Learning, DRL)的优化方法逐渐成为研究热点。深度强化学习通过模拟智能体与环境的交互,通过奖励反馈学习最优策略,能够在复杂且动态的场景中自适应调整决策,展示出强大的适应能力和高效性,如 AlphaGo Zero^[10]、Atari 等^[11]。Jia 等^[12]结合蒙特卡洛树搜索与改进 Actor-Critic 算法,引入启发式规则提升了策略鲁棒性。Zhao 等^[13]提出约束 DRL,利用“预测-投影”机制确保存放满足物理约束;其后续研究^[14]进一步利用图注意力网络解决了连续解空间下的装载问题。Yang 等^[15]通过集成启发式规则与加权体系素网格,有效优化了智能体的训练速度。Wang 等^[16]设计基于空间残值的奖励函数,帮助模型更高效地学习装箱策略。Tsang 等^[17]采用双层 DQN 结合启发式策略来筛选最佳位置,提高了空间利用率。Xiong 等^[18]通过在 DRL 中引入静态稳定性约束,实现了兼顾空间利用率的高可靠性装箱。此外,Que 等^[19]与 Xiong 等^[20]均引入了 Transformer 机制,前者构建了端到端框架,提升了泛化能力,后者提出了 GOPT 框架,利用放置生成器优化异构物体装填,显著提升了装箱效能。Almanakly 等^[21]提出了基于图像的深度强化学习方法,利用视觉表征有效优化了在线三维装箱策略。

现有的在线三维装箱方法多基于网格空间的离散化处理,导致动作空间随分辨率呈指数级增长,严

重限制了算法在连续空间下的求解效率。尽管部分研究尝试引入缓冲区或重排策略以增强灵活性,但这些方法往往缺乏系统性的规划机制:货物选择存在较大的随机性,且重排操作通常仅在空间浪费严重时被动触发,未能将“移除-重排”策略与全局序列规划进行深度协同,难以修正早期的次优决策。

针对上述局限,本研究提出了一种融合树结构搜索与深度强化学习(DRL)的分层耦合优化框架。该框架引入缓冲区与临时备份空间,构建了基于树结构的移动序列规划器,将动作决策与空间落位问题进行解耦。具体而言,外层树结构利用 DRL 提供的价值评估引导搜索方向,负责生成最优动作序列;内层 DRL 智能体则专注于连续空间内的落位优化。该方法不仅有效规避了因操作维度增加而导致的计算空间爆炸问题,而且实现了从“被动适应”到“主动规划”的跨越,显著提升了在线装箱在复杂约束下的空间利用率与决策鲁棒性。

1 问题描述

在在线三维装箱问题中,货物依次通过传送带进行运输,当到达传送带末端时,利用机械臂将其搬运至容器内。在这个场景下,本文引入缓冲区和临时备份空间,如图 1 所示。缓冲区允许智能体从多个即时货物中进行选择,改变了传统单货物选择的局限。智能体可综合考虑多个货物,这使状态表示和动作空间得以扩大。在实际装箱任务中,智能体可以根据缓冲区中不同货物的尺寸、形状等因素,选择更合适的货物进行装箱,从而提高装箱性能。如图 2 所示,临时备份空间允许移除已经装载的货物。当将许多货物装载到一个容器时,过去的放置可能会限制最终的空间利用率。因此,临时备份空间的引入对适应新的信息和改善装载结构起着至关重要的作用。

在线三维装箱问题是将给定的任意尺寸 $\mathbf{d}=[d_x, d_y, d_z]$ 的规则矩形货物装载到任意尺寸 $\mathbf{D}=[D_x, D_y, D_z]$ 的矩形容器中,目标是最大化空间利用率。给定的货物表示为元组 $\mathbf{n}=[d_n, p_n^F]$,其中 d_n 表示货物尺寸大小, p_n^F 表示货物在容器内的位置。缓冲区的货物集合用 δ 表示,临时备份空间中的货物集合用 ε 表示。每个集合的大小对应于每个区域中可以放置的货物的最大数量,分别记为 I 和 E 。这些最大容量是预先确定的参数。

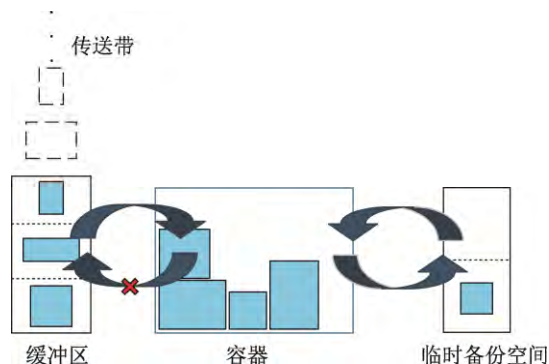


图1 引入缓冲区和临时备份空间的
装载环境示意图

Fig.1 Schematic diagram of loading
environment introducing buffer and
temporary backup space

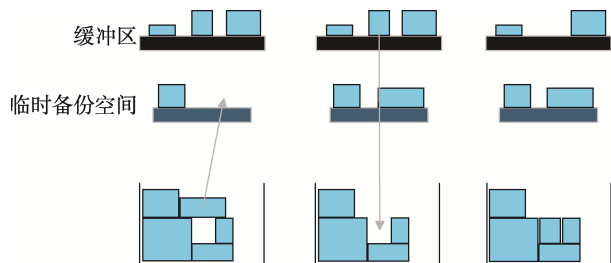


图2 移除操作示意图

Fig.2 Diagram of removal operation

将容器的内部状态抽象为有向图，用 σ 表示，如图3所示。在此图 σ 中，容器中的每个货物都对应一个节点，货物之间的支持关系对应于边。因此，很容易识别可能被删除的货物集，将其表示为 M ，红色显示的节点集对应于 M 。

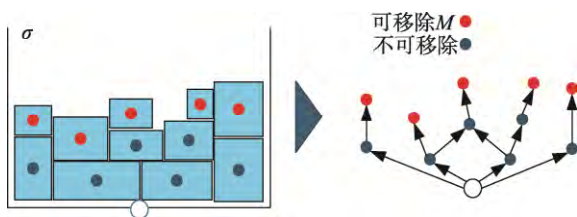


图3 有向图表示

Fig.3 Directed graph representation

表示货物 n 的装载或移除操作的动作被表示为 $a_n = \langle d_n, p_n^F, p_n^T \rangle$ ，其中 p_n^T 是货物所在的位置。动作序列表示为 $\alpha = [a_1, \dots, a_k, \dots, a_{|\alpha|}]$ ，它是从状态 S_t 执行到状态 S_{t+1} 的一系列动作。 α 的最大长度由参数 $k_{\max}$ 预先确定。本文将引入缓冲区和临时备份空间的在线三维装箱的规划问题描述为：在时间步长 t 内找到最优动作序列 α_t^* ，以最大化空间利用率。空间利用率指标为 R ，其公式如下：

$$R = \frac{\sum_{n \in \sigma} d_{n,x} d_{n,y} d_{n,z}}{D_x D_y D_z} \quad (1)$$

为使货物更加平稳码放，还需满足以下7个约束条件：1) 支撑约束，在码垛及运输过程中，货物的稳定性极为重要，确保货物稳定性的关键在于码放时货物底部要保证有充分或部分有效支撑；2) 方向约束，当执行实际码垛操作时，每件货物的空间摆放姿态必须依照预先设定的6个标准放置方向进行调整，货物的6个标准放置方向如图4所示。

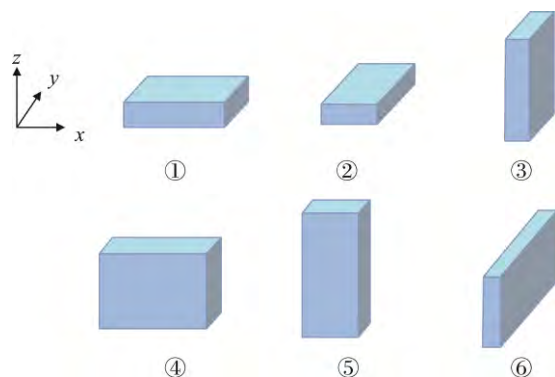


图4 方向约束

Fig.4 Directional constraints

3) 边界约束，在装载货物时，货物必须完全位于容器所限制的空间内；4) 平行约束，在货物码垛作业中，每一层货物的摆放平面必须与容器的内壁面保持严格平行对齐；5) 重合约束，容器内任意2件货物的三维空间占据区域必须完全独立，禁止出现空间重叠；6) 体积约束，货物的总体积不超过容器的最大装载体积；7) 货物装载移除约束，在在线三维装箱中，由于货物实时到达且顺序不可预知，传统方式无法对货物装载移除进行调整，通过引入缓存区与临时备份空间机制，在缓存区与临时备份空间容量限制范围内对货物的装载移除进行适当调整，从而有效提升容器的空间利用率。

2 数学模型

2.1 三维空间表示

如图5所示，建立容器的空间直角坐标系，将容器的左下前角点设为坐标系原点，货物码放的基准位置位于该原点。坐标系的 x 轴、 y 轴和 z 轴分别对应容器的长度、宽度和高度。该坐标系用于精确描述容器内部的三维空间，用货物的左下前角点描述货物的位置。在图5中，当货物的角点位置与容器原点完全重合时，其角点坐标可表示为 $(0,0,0)$ 。货物的6种码垛方向见图4。

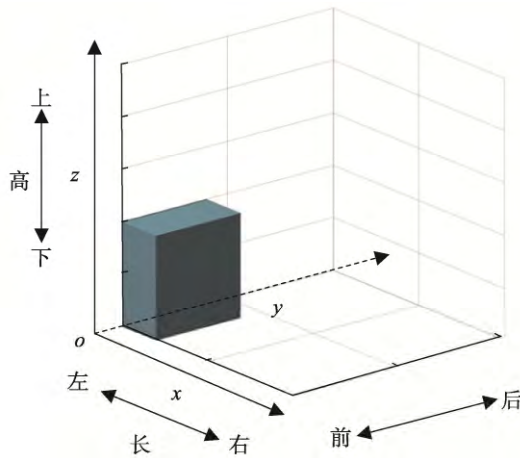


图 5 空间直角坐标系与货物位置示意图
Fig.5 Schematic diagram of space right-angle coordinate system and cargo position

2.2 剩余空间表示

货物码放后,其所占的空间将被占用,为了后续更高效的空间管理以及减少智能体的计算复杂度,采用最大子空间表示法 (Empty Maximal Space, EMS)。基于最大子空间表示法的空间划分策略能够通过 3 个相互正交的极大化子空间表征剩余可行区域。这 3 个子空间分别沿货物 x 轴延伸方向 (右部空间)、 y 轴延伸方向 (后部空间) 和 z 轴延伸方向 (上部空间) 生成,其几何边界由容器原始空间与已码放货物的几何位置共同确定,如图 6 所示。

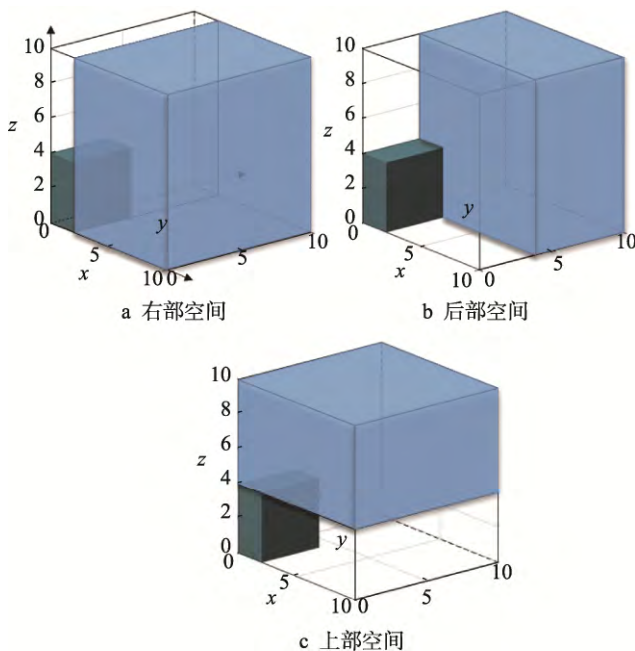


图 6 “最大子空间表示法”的分割空间示意图
Fig.6 Schematic diagram for partitioned space of "the maximum subspace representation"

2.3 符号说明

在线三维装箱建模所需符号及其含义如表 1 所示。

表 1 在线三维装箱问题建模所需符号及含义
Tab.1 Symbols and meanings required for online three-dimensional packing problem modeling

符号	含义
D_x, D_y, D_z	容器的长、宽、高
x, y, z	货物空间位置三维坐标
g_x^i, g_y^i, g_z^i	当前货物的重心坐标
d_{xi}, d_{yi}, d_{zi}	当前货物的长、宽、高
α	货物可旋转放置方向
$X_i^\alpha, Y_i^\alpha, Z_i^\alpha$	坐标轴方向实际码垛点集合
$d_i^\alpha \in \{0,1\}$	当前货物能否按方向 α 进行码放
q_{ixyz}^α	装箱动作是否将当前货物沿方向 α 码放至空间坐标点 (x, y, z) 处

2.4 模型搭建

基于格点位置范式,通过数学式描述三维装箱过程中的空间约束条件,在此基础上建立在线三维装箱问题的数学模型,具体表达如下:

$$\begin{aligned} \max \quad & \sum_{\{\alpha=1,\dots,6\}} \sum_{\{i=1,\dots,m\}} \sum_{x \in X_i^\alpha} \sum_{y \in Y_i^\alpha} \sum_{z \in Z_i^\alpha} d_{xi} \cdot d_{yi} \cdot d_{zi} \cdot q_{ixyz}^\alpha \\ \text{s.t.} \quad & \sum_{\{\alpha=1,\dots,6\}} \sum_{\{i=1,\dots,m\}} \sum_{\{x \in X_i^\alpha | s-l_i^\alpha+1 \leq x \leq s\}} \\ & \sum_{\{y \in Y_i^\alpha | t-w_i^\alpha+1 \leq y \leq t\}} \sum_{\{z \in Z_i^\alpha | u-h_i^\alpha+1 \leq z \leq u\}} \\ & q_{ixyz}^\alpha \leq 1, s \in X, t \in Y, u \in Z \\ & q_{ixyz}^\alpha \leq d_i^\alpha, i=1,\dots,m, \alpha=1,\dots,6 \end{aligned} \quad (2)$$

$$q_{ixyz}^\alpha \leq d_i^\alpha, i=1,\dots,m, \alpha=1,\dots,6 \quad (3)$$

$$\begin{cases} D_{xij}^\beta = \min(x + d_{xi}^\alpha, x' + d_{yj}^\beta) - \max(x, x') \\ D_{yij}^\beta = \min(y + d_{xi}^\alpha, y' + d_{yj}^\beta) - \max(y, y') \\ i=1,\dots,m, \alpha=1,\dots,6, \beta=1,\dots,6, x \in X_i^\alpha, y \in Y_i^\alpha, z \in Z_i^\alpha \end{cases} \quad (4)$$

$$\begin{cases} \text{range } x_1 \leq g_x^i \leq \text{range } x_2 \\ \text{range } y_1 \leq g_y^i \leq \text{range } y_2 \\ g_z^i \leq \text{range } z \end{cases} \quad (5)$$

$$\sum_{\{\alpha=1,\dots,6\}} \sum_{x \in X_i^\alpha} \sum_{y \in Y_i^\alpha} \sum_{z \in Z_i^\alpha} q_{ixyz}^\alpha \leq 1, i=1,\dots,m \quad (6)$$

$$\begin{aligned} q_{ixyz}^\alpha &\in \{0,1\}, i=1,\dots,m, \alpha=1,\dots,6, \\ x &\in X_i^\alpha, y \in Y_i^\alpha, z \in Z_i^\alpha \end{aligned} \quad (7)$$

本文在线三维装箱的优化目标为容器空间利用率最大化,如式 (1) 所示;式 (4) ~ (5) 为 C1 约束,即货物码垛时底部需满足部分支撑约束,允许一定范围内的悬空放置;式 (3) 对应 C2 约束,要求码

垛时严格遵守货物预设的6种码垛方向。C3、C4约束通过表1中符号 X_i^α 、 Y_i^α 、 Z_i^α 的定义隐含体现。式(2)为C5约束,表示放入容器中的货物所占空间之间不允许出现重合的现象。式(6)限定每件货物仅允许一次码垛;式(7)表示决策变量 q_{ixyz}^α 为0~1布尔变量,并明确参数的取值范围。

3 在线三维装箱算法

3.1 马尔可夫决策过程

对于引入缓冲区与临时备份空间的在线三维装箱场景,将其在时间 i 时刻的装载动作建模为马尔可夫决策过程(MDP),MDP模型定义如下:

状态 $\hat{s}_i$ 表示为 $\hat{s}_i = (H_i, \sigma_i, \delta_i, \varepsilon_i, d_{n_i})$,其中 H_i 为时刻 i 容器的高度图, σ_i 为时刻 i 容器的状态, δ_i 为时刻 i 缓冲区的状态, ε_i 为时刻 i 临时备份空间的状态, d_{n_i} 为时刻 i 货物的尺寸。

动作 $\hat{a}_i$ 表示为 $\hat{a}_i = (o_i, n_i, p_{n_i}^T)$,其中 o_i 为时刻 i 执行的操作类型(包含装载、移除和无操作), n_i 为时刻 i 选择的目标货物,对于装载操作, n_i 为时刻 i 传送带、缓冲区和临时备份空间中的目标货物,对于移除操作, n_i 为容器中的目标货物, $p_{n_i}^T$ 为目标货物的装载位置,其值由EMS方法生成的放置候选中选取的前左下点确定。

状态转移概率 $P(\hat{s}_{i+1} | \hat{s}_i, \hat{a}_i)$,在给定状态 $\hat{s}_i$ 与动作 $\hat{a}_i$ 后,当前货物的放置结果以及容器内空间更新过程是确定的。系统的不确定性仅来源于下一时刻到达货物的尺寸随机性。因此,状态转移概率可表示为:

$$P(\hat{s}_{i+1} | \hat{s}_i, \hat{a}_i) = \int_B P(\hat{s}_{i+1} | \hat{s}_i, \hat{a}_i, \hat{b}_{i+1}) p(\hat{b}_{i+1}) d\hat{b}_{i+1} \quad (8)$$

式中: $p(\hat{b}_{i+1})$ 表示货物尺寸的生成分布; B 为货物尺寸空间。给定 $\hat{s}_i, \hat{a}_i$ 及 $\hat{b}_{i+1}$ 后,下一状态 $\hat{s}_{i+1}$ 可唯一确定。

在本文的实验设置中,假设各时间步到达货物的尺寸相互独立且服从相同分布,其三维尺寸满足:

$$\begin{aligned} d_x &\sim \mu(l_{\min}, l_{\max}), d_y \sim \mu(w_{\min}, w_{\max}), \\ d_z &\sim \mu(h_{\min}, h_{\max}) \end{aligned} \quad (9)$$

式中: $\mu()$ 表示离散均匀分布,取值为范围内有限个整数值。

奖励函数 $\hat{r}_i$,奖励函数定义为成功装载货物所带来的体积增量,即:

$$r_i = \begin{cases} w_t & \text{(货物成功装载)} \\ -w_t & \text{(货物成功移除)} \\ 0 & \text{(无操作)} \end{cases} \quad (10)$$

式中: w_t 为目标货物的体积。在折扣因子 γ 取接近1的情况下,一个周期内的累计奖励为:

$$\sum_{i=1}^T r_i = \sum_{i=1}^N w_t \quad (11)$$

该值等价于最终成功装载货物的总体积。因此,最大化期望累计奖励在理论上等价于最大化容器的空间利用率指标 R ,从而保证了奖励函数设计与优化目标之间的一致性。

在训练过程中,DRL智能体学习策略 $\hat{\pi}(\hat{a}_i | \hat{s}_i)$ 以最大化折扣累积奖励,并学习价值函数 $\hat{V}(\hat{s}_i)$ 来预测从状态 $\hat{s}_i$ 出发的折扣累积奖励。智能体通过ACKTR算法进行训练,并通过损失函数 L 训练,定义如下:

$$L = \alpha L_{\text{actor}} + \beta L_{\text{critic}} \quad (12)$$

$$L_{\text{actor}} = \left[\hat{r}_i + \gamma \hat{V}(\hat{s}_{i+1}) - \hat{V}(\hat{s}_i) \right] \ln \hat{\pi}(\hat{a}_i | \hat{s}_i) \quad (13)$$

$$L_{\text{critic}} = \left[\hat{r}_i + \gamma \hat{V}(\hat{s}_{i+1}) - \hat{V}(\hat{s}_i) \right]^2 \quad (14)$$

式中:参数 $\alpha = \beta = 1$; γ 为折扣因子,将其设置为0.99; L_{actor} 和 L_{critic} 分别为演员损失和评论家损失。

3.2 算法设计

算法核心由基于深度强化学习(DRL)的放置位置规划器和基于树结构的移动序列规划器组成:放置位置规划器利用DRL训练智能体,结合启发式最大子空间(EMS)方法生成候选位置,通过强化学习策略选择最优装箱位置,同时处理货物移除操作,将其暂存至临时备份空间以优化后续布局。通过马尔可夫决策过程(MDP)建模,以最大化累计奖励(即空间利用率)为目标,学习单个货物的最优放置策略,确保高效利用局部空间。图7放置位置规划器中装载和移除操作示意图。移动序列规划器以树结构递归扩展,动态选择用于执行装箱或移除操作的候选货物,通过评估函数搜索最优动作序列,平衡当前收益与未来潜力。树结构的设计避免了直接扩展DRL学习空间,通过分离放置策略学习与序列规划,降低了计算复杂度,实现了多货物的全局优化。图8为移动序列规划器扩展状态节点示意图。树的每一层代表一个时间步的决策,节点分支对应从缓冲区选择货物进行装载、从容器移除顶部货物并暂存至临时备份空间、或执行无操作等动作。通过这种树结构,算法能够遍历并评估包含多个动作的决策序列,从而找到最优动作路径。

针对在线三维装箱问题中“动作序列规划”与“货物位置选择”的双重复杂性,本文提出了一种分层耦合优化框架。该框架并未将两者孤立处理,而是构建了以树结构搜索为外环决策、以深度强化学习(DRL)为内环评估的协同机制,同时设计了拓扑-空间融合感知网络来支撑这一机制。

3.2.1 分层耦合优化逻辑

本文将装箱问题分解为外层序列规划与内层位置决策2个层级,两者通过价值评估函数进行深度耦

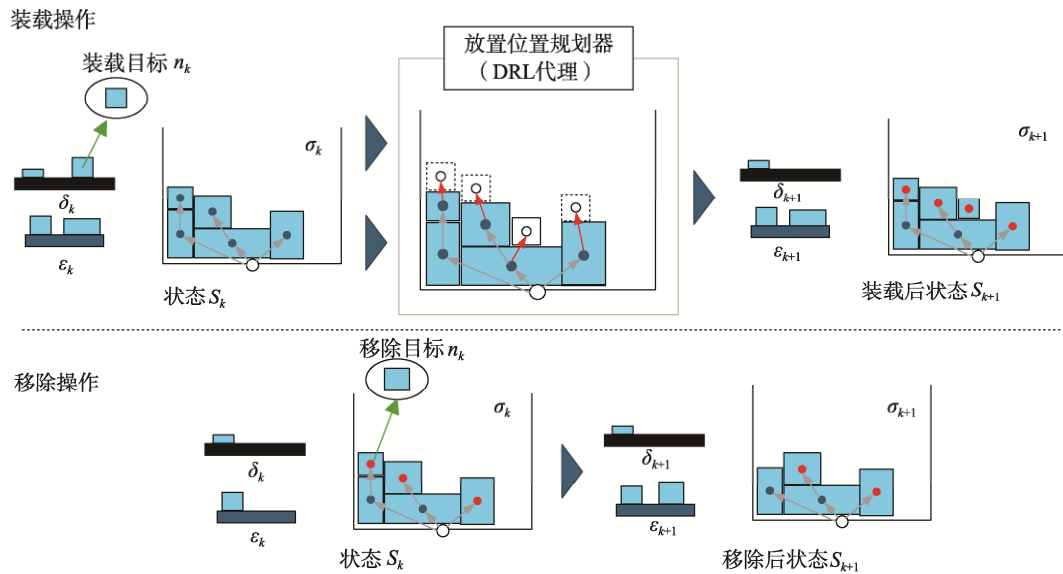


图 7 放置位置规划器中装载移除操作示意图

Fig.7 Schematic diagram of loading and removal operations in placement position planner

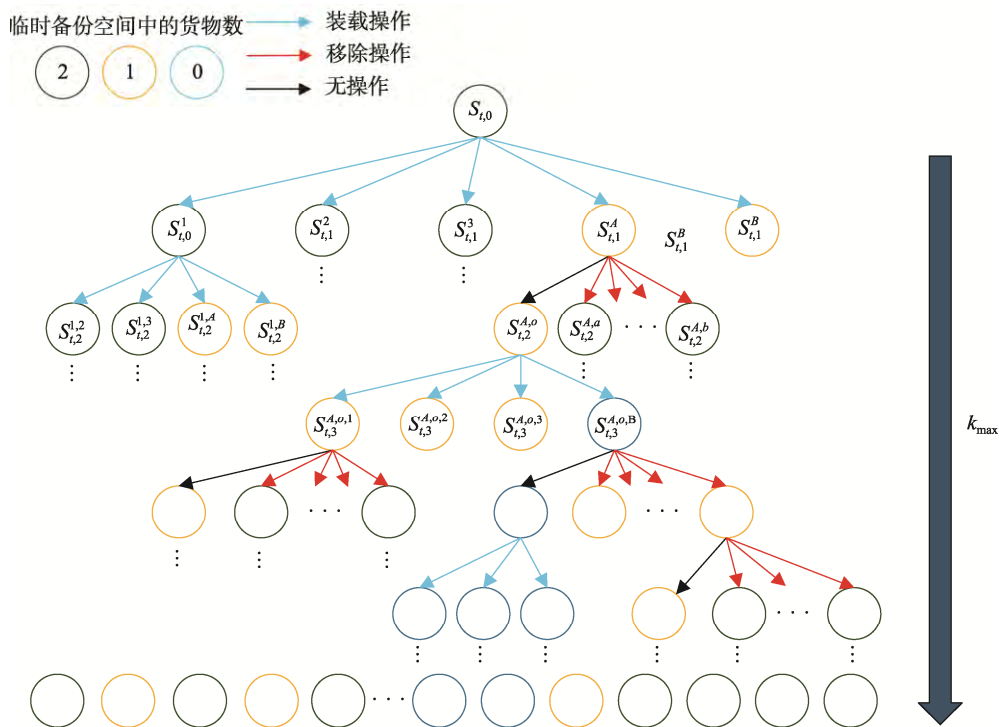


图 8 移动序列规划器扩展状态节点示意图

Fig.8 Schematic diagram for extended state node of mobile sequence planner

合。外层：引入缓冲区和临时备份空间，通过树结构移动序列规划器解决“装什么”和“移什么”的问题。该层利用树搜索算法在缓冲区和临时备份空间之间构建动作空间，生成包含货物装载、移除及无操作的动作序列 $\mathcal{a} = [a_1, \dots, a_k, \dots, a_{|\mathcal{a}|}]$ 。树结构的每一层代表一个决策步，节点的分支对应不同的货物选择或移除操作。内层：DRL 放置位置规划器解决“放在哪”的问题。对于树搜索中选定的每一个装箱动作 a_i ，调

用基于 DRL 的放置位置规划器。DRL 智能体结合 EMS 方法生成的候选点，输出最优放置位置 $p_{n_i}^F$ 。协同机制：两者的耦合体现在评估环节，树搜索不直接计算最终装箱率，而是利用 DRL 网络输出的状态价值函数 $\hat{V}(S)$ 作为启发式指引。这意味着 DRL 不仅是一个“执行者”（输出动作），更是一个“评估者”（预测当前状态的未来潜力），从而引导树搜索向长期收益更高的分支扩展，避免短视决策。

3.2.2 拓扑-空间融合感知网络

为了使 DRL 智能体能够同时理解容器内的几何空间约束和货物间的物理支撑依赖, 本文在 ACKTR 框架下设计了一种拓扑-空间融合感知网络, 构建了针对装箱特性的多模态特征提取机制, 如图 9 所示。

空间感知分支 (Spatial Perception Branch)。为了精准识别容器内的潜在落位区域, 该分支以离散高度图 H 作为输入。将容器底面划分为 $D_x \times D_y$ 的网格, 每个网格点的值对应当前堆叠的最高 z 轴坐标。该分支采用多层卷积神经网络 (CNN) 对高度图进行编码, 通过卷积核提取局部平坦度特征, 并利用最大池化层捕捉全局空间分布。输出的空间特征向量能够有效指导智能体选择能最大化空间利用率的几何位置。

拓扑感知分支 (Topological Perception Branch)。鉴于本文引入了“移除与重排”机制, 智能体必须具备对货物间支撑关系的理解能力。该分支直接利用缓冲区、临时备份空间及容器内部状态作为输入, 引入图卷积网络 (GCN) 来进行处理, 通过聚合邻居节点信息 (即支撑物与被支撑物) 生成包含结构上下文的图嵌入向量。该特征使智能体能够感知当前的堆叠结构是否稳固, 以及在执行移除操作时是否存在复杂的依赖约束, 从而有效规避“死锁”或“悬空”风险。

异构特征融合与双效输出。将空间特征、拓扑特

征以及待装货物尺寸经过多层感知器 (MLP) 进行处理, 形成综合状态表征。该表征被同时送入 Actor 和 Critic 网络: Actor 结合动态掩码输出针对 EMS 候选点的概率分布 $\pi(a|s)$, 直接驱动树节点的物理扩展; Critic 输出状态价值 $V(S)$, 该值不仅用于梯度的反向传播训练, 更关键的是被作为启发式势能项注入到树搜索的评估函数 $J(S_k)$ 中。通过 $V(S)$ 对未来装载潜力的预测, 算法能够提前识别并剪枝低价值的结构分支。

3.2.3 节点评估与引导函数

为了在树搜索中平衡即时体积收益与未来装载潜力, 本文定义了综合评估函数 $J(S_k)$ 。对于搜索树中深度为 k 的节点状态 S_k , 其评分公式为:

$$J(S_k) = \sum_{i=0}^{k-1} \gamma^i r_i + \gamma^k \hat{V}(S_k) - \lambda \sum_{i=0}^{k-1} I_{M_i}(a_{S_k, i}) \quad (15)$$

$$r_i = \begin{cases} w_t (\text{货物成功装载}) \\ -w_t (\text{货物成功移除}) \\ 0 (\text{无操作}) \end{cases} \quad (16)$$

式中: r_i 为从状态 S_i 转移到 S_{i+1} 所获得的奖励; w_t 为目标货物的体积; γ 为折扣因子; $\hat{V}(S_k)$ 为由 DRL 网络输出的状态价值估计, 其计算公式耦合了位置选择策略:

$$\hat{V}(S_k) = E_{a \sim \pi(\cdot|S_k)} [r + \gamma \hat{V}(S_{k+1})] \quad (17)$$

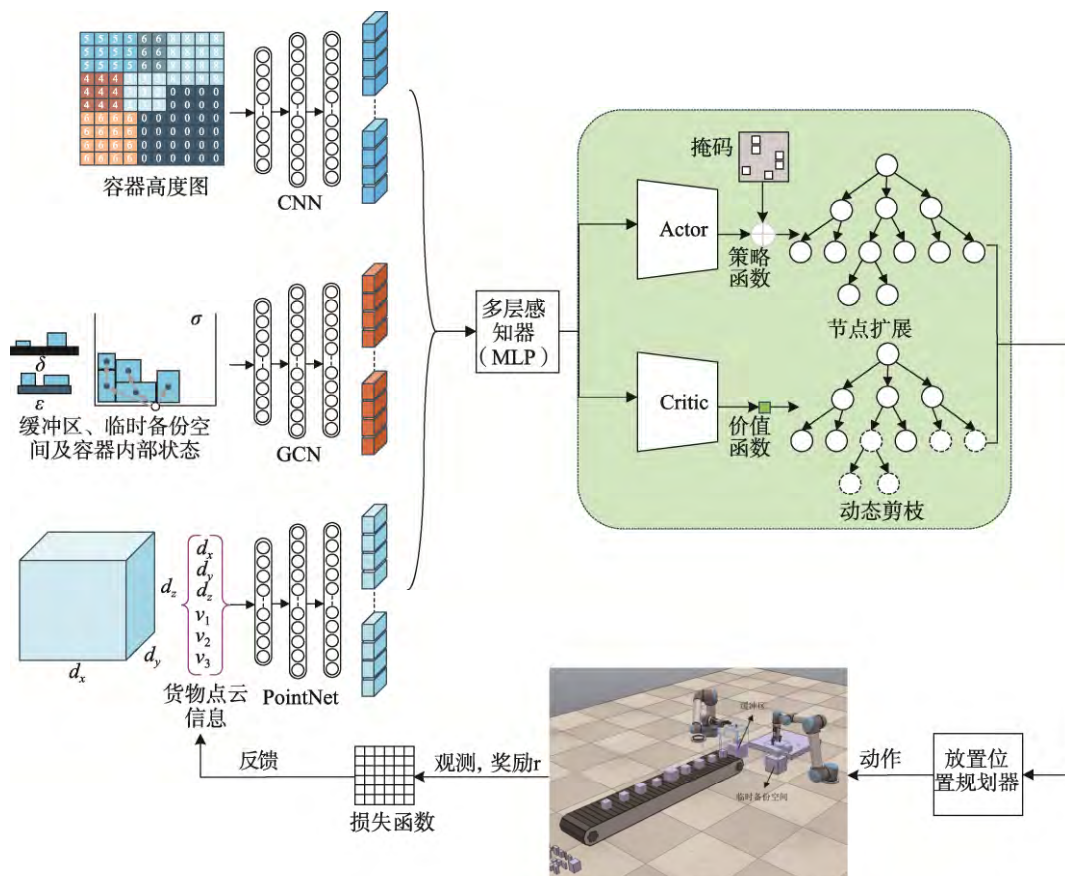


图9 在线三维装箱算法架构示意图

Fig.9 Schematic diagram of online 3D packing algorithm architecture

这意味着顶层搜索的每一次状态迁移 $S_i \rightarrow S_{i+1}$ 都是由底层 DRL 策略 $\hat{\pi}(a_i | s_i)$ 确定的最优放置位置所驱动的。这种耦合逻辑确保了序列规划是在最优位置选择的前提下进行的。

第三项为移除惩罚项, I_{M_i} 为指示函数, 当动作为移除操作时取 1, λ 为惩罚系数。当惩罚系数 λ 设置为无穷大时, 得到的操作将与临时备份空间的最大

容量 E 为 0 时的操作相同。

通过上述设计, 有效避免了直接将复杂的“移除”与“缓存”操作编码进 DRL 动作空间所导致的计算空间爆炸的问题。通过分离移动序列规划与放置位置, 算法在保证计算效率的同时, 显著提升了在复杂约束与大规模货物场景下的全局优化能力。算法流程图如图 10 所示。

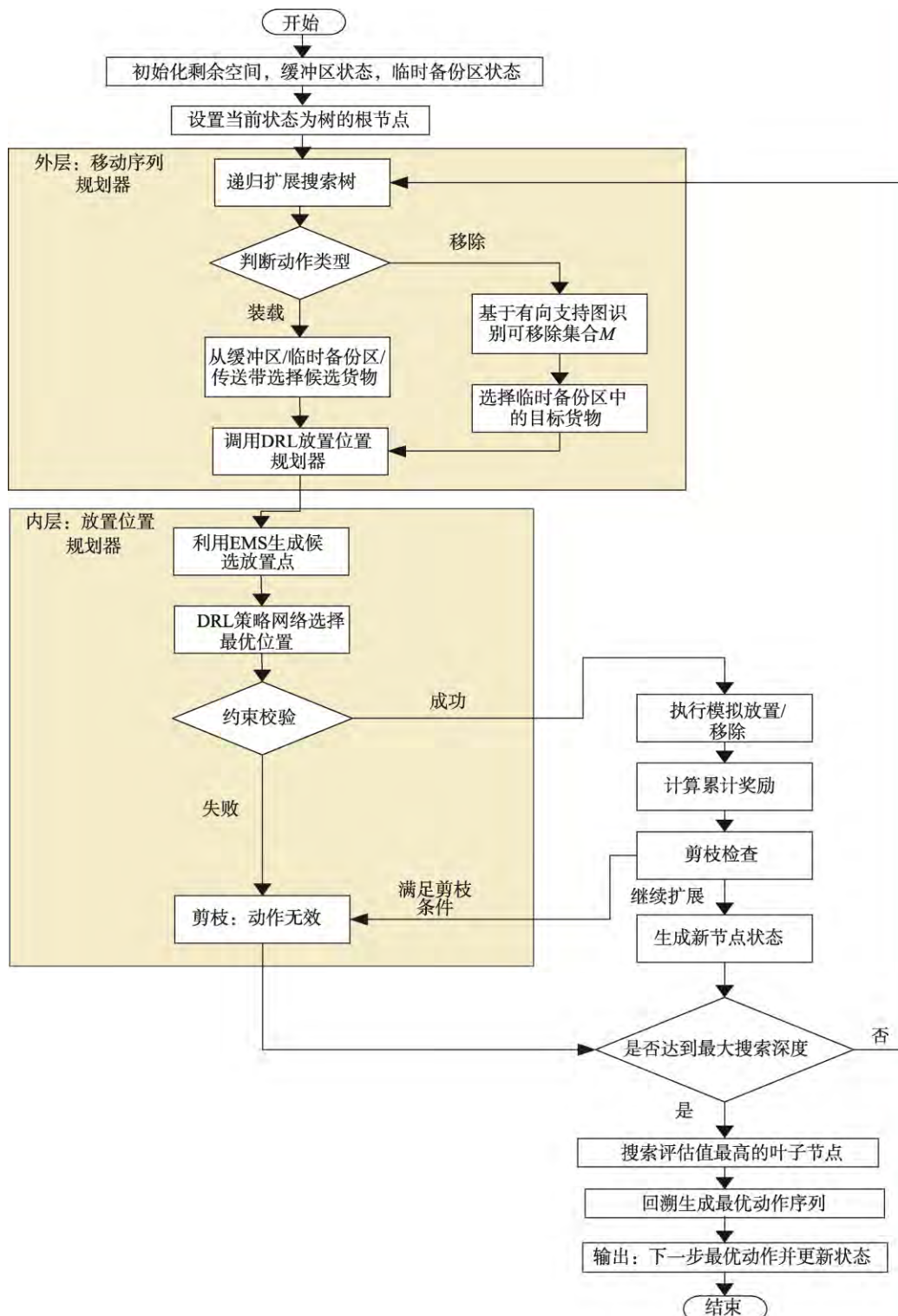


图 10 在线三维装箱算法流程图
Fig.10 Online 3D packing algorithm flowchart

3.3 树搜索加速与状态去重策略

在基于树结构的移动序列规划过程中,若不加限制地扩展搜索节点,搜索空间规模将随缓冲区容量、临时备份空间容量以及搜索深度的增加而迅速膨胀。为在保证搜索质量的前提下降低计算复杂度,本文在树搜索过程中引入基于评估函数和剩余空间上界估计的剪枝策略以及基于状态哈希的节点去重机制,以提升在线规划的效率与稳定性。

基于评估函数的剪枝策略。在树搜索过程中,每个节点对应一个装箱状态 S , 其综合评估值由累计奖励与未来期望收益共同决定, 定义为:

$$F(s) = R(s) + \gamma V(s) \quad (18)$$

式中: $R(s)$ 表示从根节点到状态 S 的累计装载奖励; $V(s)$ 为通过放置位置规划器训练得到的状态价值函数; γ 为折扣因子。

在搜索过程中, 维护当前已发现的最优评估值 F_b 。当新生成节点 S 满足式 (19) 时, 认为该节点及其后续子树不可能产生更优解, 从而终止对该节点的进一步扩展。该剪枝策略能够有效剔除低价值搜索分支, 减少无效节点扩展数量, 同时保持搜索结果的有效性。

$$F(s) < F_b \quad (19)$$

基于剩余空间上界估计的剪枝策略。为进一步提升搜索效率, 本文引入基于剩余空间上界估计的剪枝策略, 用于提前排除无法产生更优解的搜索节点。

对于任一状态 S , 设当前已装载货物体积为 $V_u(s)$, 容器剩余可用空间体积为 $V_f(s)$, 容器总体积为 V_c 。则状态 S 在理想情况下可达到的空间利用率上界定义为:

$$U(s) = \frac{V_u(s) + V_f(s)}{V_c} \quad (20)$$

该上界假设剩余空间可被完全无损填充, 因此为理论上的最大可达空间利用率。若满足式 (21), 则判定状态 S 不可能优于当前最优解, 其对应节点将被直接剪枝。该上界估计计算代价低, 适用于在线三维装箱场景中的快速搜索。

$$U(s) < F_b \quad (21)$$

基于状态哈希的节点去重机制。在引入装载与移除操作后, 不同动作序列可能导致等价的装箱状态。为避免重复搜索, 本文采用基于状态哈希的节点去重机制。

定义状态 s 的哈希表示为:

$$h(s) = H(H(s), \sigma(s), \delta(s), \varepsilon(s)) \quad (22)$$

式中: $H(s)$ 表示容器内货物的高度图; $\sigma(s)$ 表示容器内货物的有向支持关系图结构; $\delta(s)$ 表示缓冲区中的货物集合状态; $\varepsilon(s)$ 表示临时备份空间中的货物集合状态。通过维护已访问状态的哈希集合, 当新生成节点满足式 (23) 时, 认为两状态等价, 仅保留

首次访问的节点, 其余节点不再扩展。该机制有效避免了因不同操作顺序导致的状态重复扩展, 显著降低了树搜索的冗余规模, 提高了整体搜索效率。

$$h(s_i) = h(s_j), i \neq j \quad (23)$$

3.4 约束满足与冲突解决机制

在树搜索过程中, 针对移除与重新装载操作, 算法建立了严谨的约束验证流程: 1) 移除操作的稳定性校验, 算法利用有向支持图 σ 识别可移除集合 M 。仅当货物在 σ 中表现为叶子节点 (即该货物上方没有任何其他货物提供支撑) 时, 才被判定为“可移除”。这一机制从逻辑上杜绝了因移除底层货物而导致上方货物“悬空”或支撑失效的风险, 确保了容器内剩余结构的物理稳定性; 2) 重新装载的空间冲突解决, 重新装载被视为一种特殊的“装载”动作, 当货物从临时备份空间 ε 重新进入容器时, 放置位置规划器会基于当前最新的容器状态 σ_i 和 EMS 算法生成的空闲空间候选集, 重新进行位置计算; 3) 约束闭环验证: 每一个生成的动作 (无论是装载还是重装) 在状态转移前, 必须经过 7 项约束 ($C1 \sim C7$) 校验, 若某一动作无法在当前容器内找到满足支撑约束 ($C1$) 且无空间重叠 ($C5$) 的位置, 则该树分支将被标记为无效并进行剪枝, 确保所有搜索出的动作序列在物理上均是可行的。

3.5 理论分析

为了评估算法的计算效率与求解质量的可靠性, 本节从时间/空间复杂度、收敛性及剪枝误差界 3 个方面对所提框架进行理论分析。

3.5.1 复杂度分析

本文提出的算法由外层树搜索与内层 DRL 推理两部分组成。设待装箱货物总数为 n_T , 缓冲区容量为 I , 临时备份空间容量为 E , 搜索深度为 k 。

时间复杂度。在树结构的每一层, 算法需要遍历缓冲区和备份空间中的所有可行货物, 分支因子最大为 $b = I + E$ 。对于深度为 k 的搜索树, 生成的节点总数上限为:

$$\sum_{k_i=1}^k b^{k_i} = \frac{b(b^k - 1)}{b - 1} \quad (24)$$

即 $O((I + E)^k)$ 。对于每个节点, 需调用 DRL 网络生成 C_{ems} 个候选点并计算价值 $V(s)$ 。设 DRL 单次推理的时间复杂度为 $O(T_{\text{net}})$, 则单步决策的时间复杂度为 $O((I + E)^k \cdot T_{\text{net}})$ 。由于需对 n_T 个货物进行序列规划, 总时间复杂度为:

$$T_{\text{total}} = O(n_T \cdot (I + E)^k \cdot T_{\text{net}}) \quad (25)$$

该式表明, 算法耗时与搜索深度 k 呈指数关系,

与货物数量 n_T 呈线性关系。

空间复杂度。空间开销主要源于搜索树节点的存储与状态哈希表。最坏情况下需存储 $O((I+E)^k)$ 个状态节点。通过本文提出的状态哈希去重机制, 实际存储的节点数量远小于理论上限。

3.5.2 收敛性与误差界

局部最优收敛性。定义最优价值函数 $V^*(s)$ 满足贝尔曼最优方程:

$$V^*(s) = \max_{a \in A} \{R(s, a) + \gamma V^*(s')\} \quad (26)$$

本文的树搜索过程本质上是在有限深度 k 内的前瞻搜索。当搜索深度 $k \rightarrow \infty$ 且不进行剪枝时, 树搜索等价于全空间遍历, 必然收敛于全局最优解。在有限深度 k 下, 算法收敛于 k 步内的局部最优策略 π_k^* 。随着 DRL 价值网络 $\hat{V}(s)$ 的训练损失 L_{critic} , 即 $\hat{V}(s) \rightarrow V^*(s)$, 算法的决策序列将逼近全局最优解。

剪枝误差界:

由式 (20) 定义 $U(s) = V_u(s) + V_f(s) / V_c(s)$ 。其中 $V_u(s)$ 为已装载体积, $V_f(s)$ 为几何连续的剩余空闲体积。由于实际装箱受几何形状约束, 状态 s 后续能装载的最大体积 ΔV_{max} 必然满足 $\Delta V_{max} \leq V_f(s)$ 。因此, 状态 s 最终可达到的最大利用率真实值 $R_{max}(s)$ 恒满足:

$$R_{max}(s) \leq U(s) \quad (27)$$

若 $U(s) < F_b$, 则必然有 $R_{max}(s) \leq F_b$ 。因此, 属于无损剪枝。即剪除状态 s 及其子树不会丢失优于当前最优解 F_b 的解。

4 实验设计

4.1 实验环境

实验运行环境为 64 位 Ubuntu 22.04 操作系统, 选择 Python 3.8 作为开发语言, 并基于 Pytorch 1.10.0 开源机器框架开展研究。在硬件配置方面: CPU 采用 12 代英特尔酷睿 i9-12900H, 基础主频为 2.50 GHz, 配备 512 GB 内存; GPU 借助云端服务器的 NVIDIA GeForce RTX 4090 显卡, 配备 24 G 显存。

实验中, 本文选取 Zhao^[14]提出的 RS 数据集和 CUT-2 数据集来训练和评估 DRL 模型。设定容器尺寸为 (10,10,10), 货物尺寸设置为 $d_x \leq D_x/2, d_y \leq D_y/2, d_z \leq D_z/2$ 范围内随机生成, 及 $(1,1,1) \sim (5,5,5)$ 。

4.2 参数选取

本实验选用平均空间利用率 R 、平均扩展节点数量 N 以及执行移动序列规划器生成最优动作序列所需的平均计算时间 T (单位: s) 这三项核心指标来

衡量本文在线三维装箱模型的性能表现。 R 作为首要评估指标, 直观反映了模型在装箱过程中的求解能力; N 节点数量反映规划复杂度, 节点数量越多, 意味着算法在搜索空间中考虑的候选方案越全面; T 用于评估算法的实时性和计算效率。实时性约束 (T_{limit}): 在线装箱任务要求算法必须在货物到达间隔内完成决策。设定生成最优动作序列的响应时间阈值 T_{limit} 为 1 s。由于搜索树节点数 N 随 I 和 E 的增加呈指数级增长, 其计算开销必须满足: $T(I, E, k) \leq T_{limit}$ 。

4.3 实验结果及分析

表 2 为不同参数组合下的性能指标均值与标准差。实验结果显示, 随着缓冲区容量 I 和搜索深度 k 的增加, 空间利用率 R 的标准差呈现下降趋势, 这表明引入更多的搜索空间和备份机制不仅提升了装箱效率, 还增强了算法应对随机货物序列的鲁棒性。计算时间 T 的波动范围均在可控范围内, 验证了系统在动态场景下的实时稳定性。为进一步说明参数设定的工程合理性, 本文对各参数变化的敏感性进行分析。

在固定 E 与 k 的条件下, 随着缓冲区容量 I 的增加, 算法的空间利用率呈现出先显著提升、后逐渐趋于饱和的变化趋势。例如, 当 $E=1, k=3$ 时, I 从 1 增加至 2, 空间利用率由 80.2% 提升至 84.9%, 提升幅度明显; 而当 I 进一步增大时, 空间利用率的增幅逐渐减小, 但扩展节点数量与计算时间却显著增加。该现象表明, 缓冲区容量在小范围内能够有效提升货物选择灵活性, 但超过一定阈值后, 其对全局装箱结构的改善有限, 反而会因搜索空间扩大带来较高的计算开销。

临时备份空间容量 E 决定了算法可执行移除与重排操作的灵活程度。由表 2 可见, 当 $E=0$ 时, 算法退化为不允许移除的在线装箱策略, 其空间利用率明显受限; 当 E 从 1 增加至 2 时, 在相同 I 与 k 条件下, 空间利用率均有稳定提升。然而, 当 E 增大时, 扩展节点数量呈爆炸式增长, 计算时间显著上升, 且空间利用率的提升幅度逐渐减弱。这说明移除操作在小容量条件下即可发挥主要作用, 过大的临时备份空间并不能带来与其计算代价相匹配的性能收益。

搜索深度 k 直接影响树结构移动序列规划器对未来装载序列的前瞻能力。由表 2 可见, 随着 k 的增加, 空间利用率整体呈上升趋势, 但扩展节点数量与计算时间增长迅速。特别地, 当 $k>4$ 时, 计算时间已超过在线装箱系统可接受的时间范围, 而空间利用率的提升幅度相对有限。因此, 在工程应用中, 搜索深度需在性能收益与实时性之间进行折中选择。

根据表 2 结果, 当 $I=2, E=2$ 时, 算法在利用率和时间成本上达到了最优。若进一步盲目扩大阈值, 虽然理论利用率有极微小提升, 但在工程实践中会导

表 2 空间利用率、扩展节点数及计算时间性能比较
Tab.2 Comparison of space utilization, number of expansion nodes and computation time performance

参数			空间利用率 $R/\%$	扩展节点数 N	计算时间 T/s
I	E	k			
1	1	3	80.2±0.9	21.2±3.1	0.180±0.023
1	1	4	80.5±0.9	33.5±4.2	0.320±0.028
1	1	5	81.3±0.8	188.1±15.6	2.050±0.115
2	1	3	84.9±0.7	38.7±5.1	0.380±0.032
2	1	4	85.3±0.6	93.4±8.7	1.040±0.076
2	1	5	86.0±0.6	432.3±22.4	4.750±0.312
2	2	3	86.7±0.5	74.2±6.8	0.810±0.054
2	2	4	86.9±0.5	185.3±14.2	1.910±0.128
2	2	5	87.2±0.4	1651.0±85.3	14.80±0.950
3	1	3	86.2±0.6	64.2±7.4	0.720±0.045
3	1	4	86.5±0.5	131.3±9.4	2.120±0.142
3	1	5	86.9±0.4	843.6±45.2	8.320±0.520
3	2	3	88.1±0.6	124.6±11.5	1.210±0.088
3	2	4	88.4±0.5	402.7±22.1	4.080±0.245
3	2	5	89.4±0.4	2 675.0±135.8	24.300±1.650

致系统响应延迟,增加传送带停机风险,因此 $I=2$ 、 $E=2$ 是兼顾性能与稳定性的最优工程适配点。

综合以上参数敏感性分析可以看出,缓冲区容量 I 、临时备份空间容量 E 与搜索深度 k 均存在明显的性能收益拐点。在拐点之前,参数增加能够有效提升空间利用率;而超过拐点后,性能提升趋于饱和,但计算复杂度与实时性代价急剧上升。

基于上述分析,并结合在线装箱场景对实时性的要求,本文在后续实验中选取 $I=2$ 、 $E=2$ 、 $k=3$ 作为主要参数配置。该配置在保证平均规划时间小于 1 s 的前提下,实现了接近最优的空间利用率,具有良好的工程适配性。

下面选取 Zhao 等^[14]提出的在线装载配置树算法、Yang 等^[15]提出的启发式深度强化学习算法、Xiong 等^[18]提出的基于候选机制的 A2C 装箱算法以及 Xiong 等^[20]提出 GOPT 算法的作为实验对比算法。如表 3 所示,我们的算法在两种数据集上计算出的空间利用率分别为 79.6%和 86.7%,平均码放货物数量分别为 32.2 和 35.3,均高于其他算法。这归因于缓存区和临时备份空间的设定,使模型能更有效地利用空间资源。

为了验证算法在大规模货物场景下的表现,选取 Zhao 等^[14]提出的在线装载配置树算法以及 Xiong 等^[20]提出 GOPT 算法的作为实验对比算法,在 CUT-2 数据集下选取货物序列长度 n 分别为 50、100、150、200 时进行压力测试。如表 4 所示,当货物规模从 50 增加至 200 时,Zhao 等^[14]和 Xiong 等^[20]提出的算法的

空间利用率分别出现了 8.2%和 6.0%的下滑。相比之下,本文算法的空间利用率仅下降 1.4% (从 86.2%降至 84.8%)。这证明了通过引入临时备份空间与树结构搜索的模式,算法能够在长序列装箱过程中持续动态调整结构,有效避免了长距离依赖下的空间碎片化问题。

表 3 各模型在 RS、CUT-2 数据集上的性能比较
Tab.3 Performance comparison of models on RS, CUT-2 datasets

算法	RS		CUT-2	
	$R/\%$	n	$R/\%$	n
启发式算法	54.1	20.6	60.2	25.5
混合遗传算法	64.3	26.5	69.8	28.4
Zhao et al ^[14]	76.0	29.4	85.3	32.8
Yang et al ^[15]	79.3	31.8	75.1	18.2
Xiong et al ^[18]	74.6	28.4	78.3	28.8
Xiong et al ^[20]	76.1	29.5	84.6	32.6
Ours	79.6	32.2	86.7	35.3

表 4 各模型在 CUT-2 数据集上的空间利用率 R 比较
Tab.4 Space utilization (R) comparison of models on cut-2 dataset

模型	$n=50$	$n=100$	$n=150$	$n=200$
Zhao et al ^[14]	83.8%	81.5%	78.3%	75.6%
Xiong et al ^[20]	84.3%	82.5%	80.1%	78.3%
Ours	86.2%	85.7%	85.3%	84.8%

为系统评估算法在实际物流场景中的适配性,本研究量化了不同货物规模下算法生成最优动作序列平均响应时间。

为了进一步验证算法在实际物流动态环境中的实时响应能力,在 CUT-2 数据集上进行实验,在货物序列长度 n 分别为 50、100、150、200 时,验证不同货物序列长度下搜索树规划出最优动作序列的响应时间。如图 11 所示,当货物数量从 50 增加到 200 时,生成最优动作序列的耗时仅从 0.81 s 增加到 0.94 s。这得益于状态哈希去重机制,该机制将搜索空间中的冗余节点剔除。实验结果证明了本算法在保证高装箱率的同时,具备极强的实时响应能力,能够适配高动态的物流仓储环境。

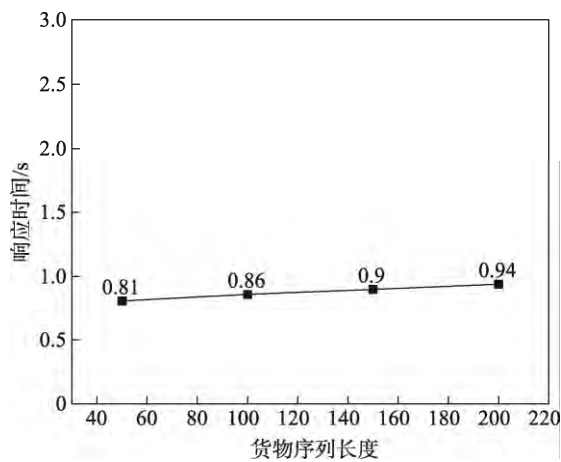


图 11 不同货物序列下的最优动作序列
响应时间曲线
Fig.11 Optimal action sequence response time
curves for different goods sequences

以 CUT-2 数据集的装载效果如图 12 所示,货物的尺寸及码垛位置如表 3 所示。

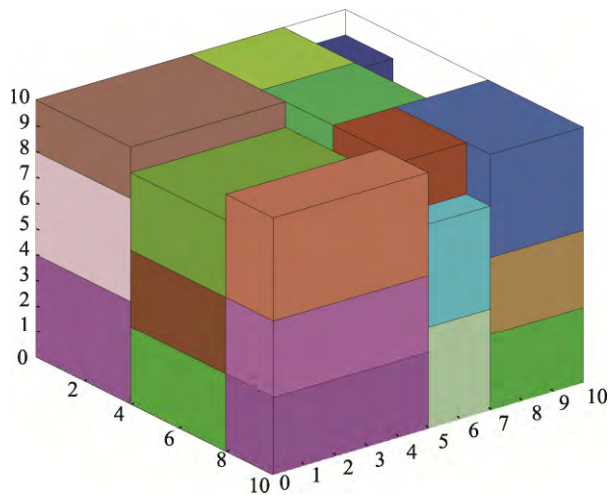


图 12 装载效果
Fig.12 Loading effect

表 5 生成的货物尺寸及码垛位置
Tab.5 Generated goods dimensions and
palletizing positions

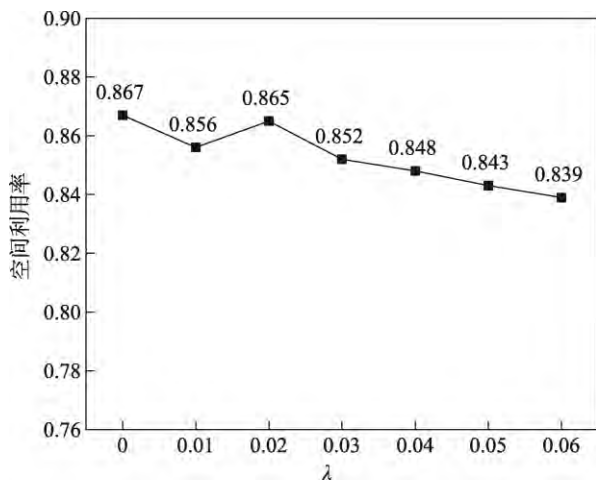
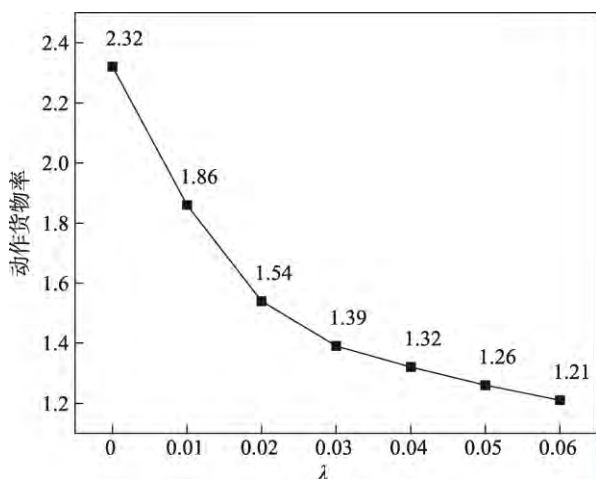
货物序号	货物尺寸/dm	码垛位置
1	(5,4,4)	(0,0,0)
2	(3,4,5)	(5,0,0)
3	(1,4,5)	(8,0,0)
...	...	...
33	(3,3,2)	(5,3,8)
34	(2,2,4)	(5,8,4)
35	(3,4,4)	(7,6,6)

4.4 移除惩罚

为了评估移除惩罚机制的有效性,本研究使用“动作货物率”作为关键性能指标。动作货物率是指装载过程中总操作次数与最终装载货物数量的比值。在未引入临时备份空间的情况下,动作货物率始终为 1。

为评估移除惩罚系数 λ 对算法性能的影响,并验证参数设置的合理性,本文在固定缓冲区容量、临时备份空间容量及搜索深度的条件下,对 λ 进行参数敏感性分析。实验中, λ 以 0.01 为步长变化,分别统计不同 λ 取值下的空间利用率以及动作货物率等关键性能指标,以分析惩罚系数变化对算法行为模式及装载效率的影响。图 13 和图 14 分别给出了惩罚系数 λ 对空间利用率和动作货物率的影响曲线。从实验结果可以观察到,惩罚系数 λ 对算法性能具有显著的调节作用。当 $\lambda=0$ 时,评估函数中不对移除操作进行惩罚,树搜索倾向于通过频繁移除与重排来探索更多潜在装载结构,虽然在一定程度上有利于空间重构,但会导致动作货物率显著升高,增加无效操作比例。随着 λ 的逐步增大,移除操作在评估函数中的代价被放大,搜索过程开始抑制收益有限的移除行为,仅在预期空间收益明显时才执行重排操作。这一机制使算法在保持较高空间利用率的同时,有效降低了动作序列长度与操作复杂度。当 λ 继续增大至较高水平时,移除操作几乎被完全抑制,算法行为逐渐退化为无临时备份空间的装载策略,空间利用率提升趋于饱和甚至略有下降。这表明过大的惩罚系数会限制算法对复杂装载结构的优化能力。

综合图 13 中的空间利用率与图 14 中的动作货物率两项指标可以发现,当 $\lambda=0.02$ 时,动作货物率从 2.32 下降至 1.54,同时保持较高的空间利用率。因此,本文将 λ 固定设置为 0.02。合理设置惩罚系数可以在保持空间利用率的同时减少更多的无效操作,从而提高装载效率。该机制为在线三维装载问题中的操作优化提供了解决方案。

图 13 惩罚系数 λ 对空间利用率的影响Fig.13 Effect of penalty coefficient λ on space utilization rate图 14 惩罚系数 λ 对动作货物率的影响Fig.14 Effect of penalty coefficient λ on action to cargo rate

5 结论

1) 创新性地提出了“序列规划-位置决策”的分层耦合架构。利用树结构在缓冲区与临时备份空间中进行宏观序列规划,赋予了算法在在线场景下的“后悔”与回溯能力;利用 DRL 解决微观定位与评估问题。该架构成功将复杂的装箱问题解耦,在不增加 DRL 状态空间维度的前提下,显著提升了算法处理长序列、大规模货物时的全局优化能力。

2) 实验结果表明,在包含支撑约束、稳定性约束等复杂现实条件下,该算法较现有主流 DRL 方法在空间利用率上提升了约 15%。特别是在引入移除惩罚机制后,算法能够在动作效率与空间利用率之间取得良好平衡,在保证平均规划时间小于 1 s 的同时,有效避免了冗余操作。

3) 尽管当前算法在仿真环境中表现优异,但距

离实际工程落地仍有优化空间。未来的研究将集中在以下 2 个方向:虚实迁移 (Sim-to-Real), 在算法中进一步集成真实物流场景的物理参数,并搭建实体实验平台进行验证。异构场景拓展,探索算法在处理异形容器、易碎品堆叠等更复杂约束场景下的适应性,进一步增强模型的泛化能力。

参考文献:

- [1] WRÓBEL S. Constraint Programming Methods in Three-Dimensional Container Packing[EB/OL]. ArXiv, 2023[2026-02-11] <https://arxiv.org/abs/2311.06314>.
- [2] FEKETE S P, SCHEPERS J, VAN DER VEEN J C. An Exact Algorithm for Higher-Dimensional Orthogonal Packing[J]. Operations Research, 2007, 55(3): 569-587.
- [3] DELL'AMICO M, FURINI F, IORI M. A Branch-and-Price Algorithm for the Temporal Bin Packing Problem[J]. Computers & Operations Research, 2020, 114: 104825.
- [4] 张德富, 彭煜, 张丽丽. 求解三维装箱问题的多层启发式搜索算法[J]. 计算机学报, 2012, 35(12): 2553-2561.
ZHANG D F, PENG Y, ZHANG L L. A Multi-Layer Heuristic Search Algorithm for Three Dimensional Container Loading Problem[J]. Chinese Journal of Computers, 2012, 35(12): 2553-2561.
- [5] SANGCHOO LI A S, SAJADIFAR S M. A Heuristic and GRASP Algorithm for Three-Dimensional Multiple Bin-Size Bin Packing Problem Based on the Needs of a Spare-Part Company[J]. International Journal of Services and Operations Management, 2021, 38(1): 73.
- [6] 张长勇, 翟一鸣. 基于改进遗传算法的航空集装箱装载问题研究[J]. 北京航空航天大学学报, 2021, 47(7): 1345-1352.
ZHANG C Y, ZHAI Y M. Air Container Loading Based on Improved Genetic Algorithm[J]. Journal of Beijing University of Aeronautics and Astronautics, 2021, 47(7): 1345-1352.
- [7] ZHANG B L, YAO Y, KAN H K, et al. A GAN-Based Genetic Algorithm for Solving the 3D Bin Packing Problem[J]. Scientific Reports, 2024, 14: 7775.
- [8] 刘胜, 沈大勇, 商秀芹, 等. 求解三维装箱问题的多层树搜索算法[J]. 自动化学报, 2020, 46(6): 1178-1187.
LIU S, SHEN D Y, SHANG X Q, et al. A Multi-Level Tree Search Algorithm for Three Dimensional Container Loading Problem[J]. Acta Automatica Sinica, 2020, 46(6): 1178-1187.
- [9] 邢志伟, 侯翔开, 李彪, 等. 基于动态四叉树搜索的

- 民航行李车码放算法[J]. 北京航空航天大学学报, 2022, 48(12): 2345-2355.
- XING Z W, HOU X K, LI B, et al. Civil Aviation Luggage Cart Stacking Algorithm Based on Dynamic Quadtree Search[J]. Journal of Beijing University of Aeronautics and Astronautics, 2022, 48(12): 2345-2355.
- [10] SILVER D, SCHRITTWIESER J, SIMONYAN K, et al. Mastering the Game of Go without Human Knowledge[J]. Nature, 2017, 550(7676): 354-359.
- [11] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-Level Control through Deep Reinforcement Learning[J]. Nature, 2015, 518(7540): 529-533.
- [12] JIA J, SHANG H L, CHEN X. Robot Online 3D Bin Packing Strategy Based on Deep Reinforcement Learning and 3D Vision[C]// 2022 IEEE International Conference on Networking, Sensing and Control (ICNSC). Shanghai, China. IEEE, 2023: 1-6.
- [13] ZHAO H, SHE Q J, ZHU C Y, et al. Online 3D Bin Packing with Constrained Deep Reinforcement Learning[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2021, 35(1): 741-749.
- [14] ZHAO H, YU Y, XU K. Learning Efficient Online 3d Bin Packing on Packing Configuration Trees[J]. International Conference on Learning Representations, 2021, 1(1): 1-18.
- [15] YANG S, SONG S, CHU S L, et al. Heuristics Integrated Deep Reinforcement Learning for Online 3D Bin Packing[J]. IEEE Transactions on Automation Science and Engineering, 2024, 21(1): 939-950.
- [16] WANG B Y, LIN Z H, KONG W J, et al. Bin Packing Optimization via Deep Reinforcement Learning[J]. IEEE Robotics and Automation Letters, 2025, 10(3): 2542-2549.
- [17] TSANG Y P, MO D Y, CHUNG K T, et al. A Deep Reinforcement Learning Approach for Online and Concurrent 3D Bin Packing Optimisation with Bin Replacement Strategies[J]. Computers in Industry, 2025, 164: 104202.
- [18] XIONG H, DING K, DING W, et al. Towards Reliable Robot Packing System Based on Deep Reinforcement Learning[J]. Advanced Engineering Informatics, 2023, 57: 102028.
- [19] QUE Q Q, YANG F, ZHANG D F. Solving 3D Packing Problem Using Transformer Network and Reinforcement Learning[J]. Expert Systems with Applications, 2023, 214: 119153.
- [20] XIONG H, GUO C R, PENG J, et al. GOPT: Generalizable Online 3D Bin Packing via Transformer-Based Deep Reinforcement Learning[J]. IEEE Robotics and Automation Letters, 2024, 9(11): 10335-10342.
- [21] ALMANAKLY H. Online 3D Bin Packing an Image-Based Deep Reinforcement Learning Approach[D]. New York: The Cooper Union for the Advancement of Science and Art, 2025.